

РБНФ №1 (опис синтаксису мови всіма допустимими засобами РБНФ)	РБНФ №2 (опис формальна граматика мови)	Опис формальної граматики мови	Опис граматики з специфікацією lookahead для LL2-аналізатора	/* Код для перевірки РБНФ №1 */	/* Код для перевірки РБНФ №2 */	/* Дані для LL2-аналізатора (лексми вже опрацьовані лексичним аналізатором) УВАГА: при копіюванні зважайте, щоб у кожному рядку після символу «\» не містилось жодних інших символів. */
		G = (N, T, P, S)	G = (N, T, P, S)			
		S → program_rule	S → program_rule			
		N = { program_name, value_type, array_specify, declaration_element, array_specify__optional, other_declaration_ident, declaration, other_declaration_ident__iteration, index_action, unary_operator, unary_operation, binary_operator, binary_action, left_expression, group_expression, index_action__optional, expression, binary_action__iteration, expression_or_cond_block__with_optional_assign, assign_to_right, assign_to_right__optional, if_expression, body_for_true, false_cond_block_without_else, body_for_false, cond_block, false_cond_block_without_else__iteration, body_for_false__optional, cycle_begin_expression, cycle_end_expression, cycle_counter, cycle_counter_lr_init, cycle_counter_init, cycle_counter_last_value, cycle_body, forto_cycle, statement, statement__or__block_statements, block_statements, input_rule, argument_for_input, output_rule, statement__iteration, expression__optional, program_rule, declaration__optional, non_zero_digit, digit__iteration, digit, unsigned_value, value, sign__optional, sign, ident, letter_in_upper_case, letter_in_lower_case, sign_plus, sign_minus }	N = { program_name, value_type, array_specify, declaration_element, array_specify__optional, other_declaration_ident, declaration, other_declaration_ident__iteration, index_action, unary_operator, unary_operation, binary_operator, binary_action, left_expression, group_expression, index_action__optional, expression, binary_action__iteration, expression_or_cond_block__with_optional_assign, assign_to_right, assign_to_right__optional, if_expression, body_for_true, false_cond_block_without_else, body_for_false, cond_block, false_cond_block_without_else__iteration, body_for_false__optional, cycle_begin_expression, cycle_end_expression, cycle_counter, cycle_counter_lr_init, cycle_counter_init, cycle_counter_last_value, cycle_body, forto_cycle, statement, statement__or__block_statements, block_statements, input_rule, argument_for_input, output_rule, statement__iteration, expression__optional, program_rule, declaration__optional, non_zero_digit, digit__iteration, digit, unsigned_value, value, sign__optional, sign, ident, letter_in_upper_case, letter_in_lower_case, sign_plus, sign_minus }	#define NONTERMINALS program_name, \ value_type, \ array_specify, \ declaration_element, \ \ other_declaration_ident, \ declaration, \ \ index_action, \ unary_operator, \ unary_operation, \ binary_operator, \ binary_action, \ left_expression, \ group_expression, \ \ expression, \ \ expression_or_cond_block__with_optional_assign, \ assign_to_right, \ \ if_expression, \ body_for_true, \ false_cond_block_without_else, \ body_for_false, \ cond_block, \ \ cycle_begin_expression, \ cycle_end_expression, \ cycle_counter, \ cycle_counter_lr_init, \ cycle_counter_init, \ cycle_counter_last_value, \ cycle_body, \ forto_cycle, \ statement, \ statement__or__block_statements,\ block_statements, \ input_rule, \ argument_for_input, \ output_rule, \ \ \ program_rule, \ \ non_zero_digit, \ digit__iteration, \ digit, \ unsigned_value, \ value, \ \ sign, \ ident, \ letter_in_upper_case, \ letter_in_lower_case, \ sign_plus, \ sign_minus	#define NONTERMINALS program_name, \ value_type, \ array_specify, \ declaration_element, \ array_specify__optional, \ other_declaration_ident, \ declaration, \ other_declaration_ident__iteration, \ index_action, \ unary_operator, \ unary_operation, \ binary_operator, \ binary_action, \ left_expression, \ group_expression, \ index_action__optional, \ expression, \ binary_action__iteration, \ expression_or_cond_block__with_optional_assign, \ assign_to_right, \ assign_to_right__optional, \ if_expression, \ body_for_true, \ false_cond_block_without_else, \ body_for_false, \ cond_block, \ false_cond_block_without_else__iteration, \ body_for_false__optional, \ cycle_begin_expression, \ cycle_end_expression, \ cycle_counter, \ cycle_counter_lr_init, \ cycle_counter_init, \ cycle_counter_last_value, \ cycle_body, \ forto_cycle, \ statement, \ statement__or__block_statements,\ block_statements, \ input_rule, \ argument_for_input, \ output_rule, \ statement__iteration, \ expression__optional, \ program_rule, \ declaration__optional, \ non_zero_digit, \ digit__iteration, \ digit, \ unsigned_value, \ value, \ sign__optional, \ sign, \ ident, \ letter_in_upper_case, \ letter_in_lower_case, \ sign_plus, \ sign_minus	
		T = { "INTEGER16" , " " , "NOT" , "AND" , "OR" , "==" , "!=" , "<=" , ">=" , "+" , "-" , "*" , "/" , "DIV" , "MOD" , "(" , ")" , "=" , "ELSE" , "IF" , "DO" , "FOR" ,	T = { "INTEGER16" , " " , "NOT" , "AND" , "OR" , "==" , "!=" , "<=" , ">=" , "+" , "-" , "*" , "/" , "DIV" , "MOD" , "(" , ")" , "=" , "ELSE" , "IF" , "DO" , "FOR" ,	#define TOKENS \ tokenINTEGER16, \ tokenCOMMA, \ tokenNOT, \ tokenAND, \ tokenOR, \ tokenEQUAL, \ tokenNOTEQUAL, \ tokenLESSOREQUAL, \ tokenGREATEROREQUAL, \ tokenPLUS, \ tokenMINUS, \ tokenMUL, \ tokenDIV, \ tokenMOD, \ tokenGROUPEXPRESSIONBEGIN, \ tokenGROUPEXPRESSIONEND, \ tokenLRASSIGN, \ tokenELSE, \ tokenIF, \ tokenDO, \ tokenFOR, \ 	#define TOKENS \ tokenINTEGER16, \ tokenCOMMA, \ tokenNOT, \ tokenAND, \ tokenOR, \ tokenEQUAL, \ tokenNOTEQUAL, \ tokenLESSOREQUAL, \ tokenGREATEROREQUAL, \ tokenPLUS, \ tokenMINUS, \ tokenMUL, \ tokenDIV, \ tokenMOD, \ tokenGROUPEXPRESSIONBEGIN, \ tokenGROUPEXPRESSIONEND, \ tokenLRASSIGN, \ tokenELSE, \ tokenIF, \ tokenDO, \ tokenFOR, \ 	

		"TO", "EXIT", "GET", "PUT", "NAME", "BODY", "DATA", "BEGIN", "END", "{", "}", "[", "]", ",", ";", "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", " ", "_", "A", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K", "L", "M", "N", "O", "P", "Q", "R", "S", "T", "U", "V", "W", "X", "Y", "Z", "a", "b", "c", "d", "e", "f", "g", "h", "i", "j", "k", "l", "m", "n", "o", "p", "q", "r", "s", "t", "u", "v", "w", "x", "y", "z"} 	"TO", "EXIT", "GET", "PUT", "NAME", "BODY", "DATA", "BEGIN", "END", "{", "}", "[", "]", ",", ";", "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", " ", "_", "A", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K", "L", "M", "N", "O", "P", "Q", "R", "S", "T", "U", "V", "W", "X", "Y", "Z", "a", "b", "c", "d", "e", "f", "g", "h", "i", "j", "k", "l", "m", "n", "o", "p", "q", "r", "s", "t", "u", "v", "w", "x", "y", "z"} 	tokenTO, \ tokenEXIT, \ tokenGET, \ tokenPUT, \ tokenNAME, \ tokenBODY, \ tokenDATA, \ tokenBEGIN, \ tokenEND, \ tokenBEGINBLOCK, \ tokenENDBLOCK, \ tokenLEFTSQUAREBRACKETS, \ tokenRIGHTSQUAREBRACKETS, \ tokenSEMICOLON, \ digit_0, \ digit_1, \ digit_2, \ digit_3, \ digit_4, \ digit_5, \ digit_6, \ digit_7, \ digit_8, \ digit_9, \ tokenUNDERSCORE, \ A, \ B, \ C, \ D, \ E, \ F, \ G, \ H, \ I, \ J, \ K, \ L, \ M, \ N, \ O, \ P, \ Q, \ R, \ S, \ T, \ U, \ V, \ W, \ X, \ Y, \ Z, \ a, \ b, \ c, \ d, \ e, \ f, \ g, \ h, \ i, \ j, \ k, \ l, \ m, \ n, \ o, \ p, \ q, \ r, \ s, \ t, \ u, \ v, \ w, \ x, \ y, \ z #define COMMENT_BEGIN_STR "##" #define COMMENT_END_STR "##" <
--	--	--	--	--

						#define T_END_BLOCK_3 ""
				tokenSEMICOLON = ";" >> BOUNDARIES;	tokenSEMICOLON = ";" >> BOUNDARIES;	#define T_SEMICOLON_0 ";" #define T_SEMICOLON_1 "" #define T_SEMICOLON_2 "" #define T_SEMICOLON_3 ""
				tokenINTEGER16 = "INTEGER16" >> STRICT_BOUNDARIES;	tokenINTEGER16 = "INTEGER16" >> STRICT_BOUNDARIES;	#define T_DATA_TYPE_0 "INTEGER16" #define T_DATA_TYPE_1 "" #define T_DATA_TYPE_2 "" #define T_DATA_TYPE_3 ""
				tokenCOMMA = "," >> BOUNDARIES;	tokenCOMMA = "," >> BOUNDARIES;	#define T_COMA_0 "," #define T_COMA_1 "" #define T_COMA_2 "" #define T_COMA_3 ""
						#define T_BITWISE_NOT_0 "~" #define T_BITWISE_NOT_1 "" #define T_BITWISE_NOT_2 "" #define T_BITWISE_NOT_3 ""
				tokenNOT = "NOT" >> STRICT_BOUNDARIES;	tokenNOT = "NOT" >> STRICT_BOUNDARIES;	#define T_NOT_0 "NOT" #define T_NOT_1 "" #define T_NOT_2 "" #define T_NOT_3 ""
						#define T_BITWISE_AND_0 "&" #define T_BITWISE_AND_1 "" #define T_BITWISE_AND_2 "" #define T_BITWISE_AND_3 ""
				tokenAND = "AND" >> STRICT_BOUNDARIES;	tokenAND = "AND" >> STRICT_BOUNDARIES;	#define T_AND_0 "AND" #define T_AND_1 "" #define T_AND_2 "" #define T_AND_3 ""
						#define T_BITWISE_OR_0 " " #define T_BITWISE_OR_1 "" #define T_BITWISE_OR_2 "" #define T_BITWISE_OR_3 ""
				tokenOR = "OR" >> STRICT_BOUNDARIES;	tokenOR = "OR" >> STRICT_BOUNDARIES;	#define T_OR_0 "OR" #define T_OR_1 "" #define T_OR_2 "" #define T_OR_3 ""
				tokenEQUAL = "==" >> BOUNDARIES;	tokenEQUAL = "==" >> BOUNDARIES;	#define T_EQUAL_0 "==" #define T_EQUAL_1 "" #define T_EQUAL_2 "" #define T_EQUAL_3 ""
				tokenNOTEQUAL = "!=" >> BOUNDARIES;	tokenNOTEQUAL = "!=" >> BOUNDARIES;	#define T_NOT_EQUAL_0 "!=" #define T_NOT_EQUAL_1 "" #define T_NOT_EQUAL_2 "" #define T_NOT_EQUAL_3 ""
				tokenLESSOREQUAL = "<=" >> BOUNDARIES;	tokenLESSOREQUAL = "<=" >> BOUNDARIES;	#define T_LESS_OR_EQUAL_0 "<="
				tokenGREATEROREQUAL = ">=" >> BOUNDARIES;	tokenGREATEROREQUAL = ">=" >> BOUNDARIES;	#define T_GREATER_OR_EQUAL_0 ">="
				tokenPLUS = "+" >> BOUNDARIES;	tokenPLUS = "+" >> BOUNDARIES;	#define T_ADD_0 "+" #define T_ADD_1 "" #define T_ADD_2 "" #define T_ADD_3 ""
				tokenMINUS = "-" >> BOUNDARIES;	tokenMINUS = "-" >> BOUNDARIES;	#define T_SUB_0 "-" #define T_SUB_1 "" #define T_SUB_2 "" #define T_SUB_3 ""
				tokenMUL = "*" >> BOUNDARIES;	tokenMUL = "*" >> BOUNDARIES;	#define T_MUL_0 "*"
				tokenDIV = "DIV" >> STRICT_BOUNDARIES;	tokenDIV = "DIV" >> STRICT_BOUNDARIES;	#define T_DIV_0 "DIV" #define T_DIV_1 "" #define T_DIV_2 "" #define T_DIV_3 ""
				tokenMOD = "MOD" >> STRICT_BOUNDARIES;	tokenMOD = "MOD" >> STRICT_BOUNDARIES;	#define T_MOD_0 "MOD" #define T_MOD_1 "" #define T_MOD_2 "" #define T_MOD_3 ""
				tokenLRASSIGN = "=" >> BOUNDARIES;	tokenLRASSIGN = "=" >> BOUNDARIES;	#define T_LRASSIGN_0 "=" #define T_LRASSIGN_1 "" #define T_LRASSIGN_2 "" #define T_LRASSIGN_3 ""
						#define T_THEN_BLOCK_0 "(" #define T_THEN_BLOCK_1 "" #define T_THEN_BLOCK_2 "" #define T_THEN_BLOCK_3 ""
				tokenELSE = "ELSE" >> STRICT_BOUNDARIES;	tokenELSE = "ELSE" >> STRICT_BOUNDARIES;	#define T_ELSE_BLOCK_0 "ELSE" #define T_ELSE_BLOCK_1 T_BEGIN_BLOCK_0 #define T_ELSE_BLOCK_2 "" #define T_ELSE_BLOCK_3 ""
				tokenIF = "IF" >> STRICT_BOUNDARIES;	tokenIF = "IF" >> STRICT_BOUNDARIES;	#define T_IF_0 "IF" #define T_IF_1 "" #define T_IF_2 "" #define T_IF_3 ""
						#define T_ELSE_IF_0 "ELSE" #define T_ELSE_IF_1 T_IF_0 #define T_ELSE_IF_2 "" #define T_ELSE_IF_3 ""
				tokenDO = "DO" >> STRICT_BOUNDARIES;	tokenDO = "DO" >> STRICT_BOUNDARIES;	#define T_DO_0 "DO" #define T_DO_1 "" #define T_DO_2 "" #define T_DO_3 ""
				tokenFOR = "FOR" >> STRICT_BOUNDARIES;	tokenFOR = "FOR" >> STRICT_BOUNDARIES;	#define T_FOR_0 "FOR" #define T_FOR_1 "" #define T_FOR_2 "" #define T_FOR_3 ""
				tokenTO = "TO" >> STRICT_BOUNDARIES;	tokenTO = "TO" >> STRICT_BOUNDARIES;	#define T_TO_0 "TO" #define T_TO_1 "" #define T_TO_2 "" #define T_TO_3 ""
				tokenEXIT = "EXIT" >> STRICT_BOUNDARIES;	tokenEXIT = "EXIT" >> STRICT_BOUNDARIES;	#define T_EXIT_0 "EXIT"

						#define T_EXIT_1 "" #define T_EXIT_2 "" #define T_EXIT_3 ""
				tokenGET = "GET" >> STRICT_BOUNDARIES;	tokenGET = "GET" >> STRICT_BOUNDARIES;	#define T_INPUT_0 "GET" #define T_INPUT_1 "" #define T_INPUT_2 "" #define T_INPUT_3 ""
				tokenPUT = "PUT" >> STRICT_BOUNDARIES;	tokenPUT = "PUT" >> STRICT_BOUNDARIES;	#define T_OUTPUT_0 "PUT" #define T_OUTPUT_1 "" #define T_OUTPUT_2 "" #define T_OUTPUT_3 ""
				tokenNAME = "NAME" >> STRICT_BOUNDARIES;	tokenNAME = "NAME" >> STRICT_BOUNDARIES;	#define T_NAME_0 "NAME" #define T_NAME_1 "" #define T_NAME_2 "" #define T_NAME_3 ""
				tokenBODY = "BODY" >> STRICT_BOUNDARIES;	tokenBODY = "BODY" >> STRICT_BOUNDARIES;	#define T_BODY_0 "BODY" #define T_BODY_1 "" #define T_BODY_2 "" #define T_BODY_3 ""
				tokenDATA = "DATA" >> STRICT_BOUNDARIES;	tokenDATA = "DATA" >> STRICT_BOUNDARIES;	#define T_DATA_0 "DATA" #define T_DATA_1 "" #define T_DATA_2 "" #define T_DATA_3 ""
				tokenBEGIN = "BEGIN" >> STRICT_BOUNDARIES;	tokenBEGIN = "BEGIN" >> STRICT_BOUNDARIES;	#define T_BEGIN_0 "BEGIN" #define T_BEGIN_1 "" #define T_BEGIN_2 "" #define T_BEGIN_3 ""
				tokenEND = "END" >> STRICT_BOUNDARIES;	tokenEND = "END" >> STRICT_BOUNDARIES;	#define T_END_0 "END" #define T_END_1 "" #define T_END_2 "" #define T_END_3 ""
						#define T_NULL_STATEMENT_0 "NULL" #define T_NULL_STATEMENT_1 "STATEMENT" #define T_NULL_STATEMENT_2 "" #define T_NULL_STATEMENT_3 ""
						#define GRAMMAR_LL2_2025 {\
program_name = ident;	program_name = ident;	program_name → ident	program_name(1: "ident_terminal") → ident	program_name = SAME_RULE(ident);	program_name = SAME_RULE(ident);	{ LA_IS, {"ident_terminal"}, {"program_name"}, {\
value_type = "INTEGER16";	value_type = "INTEGER16";	value_type → "INTEGER16"	value_type(1: "INTEGER16") → "INTEGER16"	value_type = SAME_RULE(tokenINTEGER16);	value_type = SAME_RULE(tokenINTEGER16);	{ LA_IS, {T_DATA_TYPE_0}, {"value_type"}, {\
	array_specify = "[" , unsigned_value , "]" ;	array_specify → "[" unsigned_value "]"	array_specify(1: "[") → "[" unsigned_value "]"		array_specify = "[" >> unsigned_value >> "]" ;	{ LA_IS, {"["], {"array_specify"}, {\
declaration_element = ident, ["[" , unsigned_value , "]"] ;	declaration_element = ident, array_specify__optional;	declaration_element → ident array_specify__optional	declaration_element(1: "ident_terminal") → ident array_specify__optional	declaration_element = ident >> -(tokenLEFTSQUAREBRACKETS >> unsigned_value >> tokenRIGHTSQUAREBRACKETS);	declaration_element = ident >> array_specify__optional;	{ LA_IS, {"ident_terminal"}, {"declaration_element"}, {\
	array_specify__optional = array_specify ε;	array_specify__optional → array_specify array_specify__optional → ε	array_specify__optional(1: "[") → array_specify array_specify__optional(1: !"[") → ε		array_specify__optional = array_specify "";	{ LA_IS, {"["], {"array_specify__optional"}, {\
other_declaration_ident = " , " , declaration_element;	other_declaration_ident = " , " , declaration_element;	other_declaration_ident → " , " , declaration_element	other_declaration_ident(1: " , ") → " , " , declaration_element	other_declaration_ident = tokenCOMMA >> declaration_element;	other_declaration_ident = tokenCOMMA >> declaration_element;	{ LA_IS, {T_COMA_0}, {"other_declaration_ident"}, {\
declaration = value_type , declaration_element , {other_declaration_ident};	declaration = value_type , declaration_element , other_declaration_ident__iteration;	declaration → value_type declaration_element other_declaration_ident__iteration	declaration(1: "INTEGER16") → value_type declaration_element other_declaration_ident__iteration	declaration = value_type >> declaration_element >> *other_declaration_ident;	declaration = value_type >> declaration_element >> other_declaration_ident__iteration;	{ LA_IS, {T_DATA_TYPE_0}, {"declaration"}, {\
	other_declaration_ident__iteration = other_declaration_ident, other_declaration_ident__iteration ε;	other_declaration_ident__iteration → other_declaration_ident other_declaration_ident__iteration false_cond_block_without_else__iteration → ε	other_declaration_ident__iteration(1: " , ") → other_declaration_ident other_declaration_ident__iteration false_cond_block_without_else__iteration(1: !" , ") → ε		other_declaration_ident__iteration = other_declaration_ident >> other_declaration_ident__iteration "";	{ LA_IS, {T_COMA_0 }, {\
index_action = "[" , expression , "]" ;	index_action = "[" , expression , "]" ;	index_action → "[" expression "]"	index_action(1: "[") → "[" expression "]"	index_action = tokenLEFTSQUAREBRACKETS >> expression >> tokenRIGHTSQUAREBRACKETS;	index_action = tokenLEFTSQUAREBRACKETS >> expression >> tokenRIGHTSQUAREBRACKETS;	{ LA_IS, {"["], {"index_action"}, {\
unary_operator = "NOT";	unary_operator = "NOT";	unary_operator → "NOT"	unary_operator(1: "NOT") → "NOT"		unary_operator = SAME_RULE(tokenNOT);	{ LA_IS, {T_NOT_0 }, {"unary_operator"}, {\
unary_operation = unary_operator , expression;	unary_operation = unary_operator , expression;	unary_operation → unary_operator expression	unary_operation(1: "NOT") → unary_operator expression	unary_operation = unary_operator >> expression;	unary_operation = unary_operator >> expression;	{ LA_IS, {T_NOT_0 }, {"unary_operation"}, {\
binary_operator = "AND" "OR" "==" "!=" "<=" ">=" "+" "-" "*" "DIV" "MOD";	binary_operator = "AND" "OR" "==" "!=" "<=" ">=" "+" "-" "*" "DIV" "MOD";	binary_operator → "AND" binary_operator → "OR" binary_operator → "==" binary_operator → "!=" binary_operator → "<=" binary_operator → ">=" binary_operator → "+" binary_operator → "-" binary_operator → "*" binary_operator → "DIV" binary_operator → "MOD"	binary_operator(1: "AND") → "AND" binary_operator(1: "OR") → "OR" binary_operator(1: "==") → "==" binary_operator(1: "!=") → "!=" binary_operator(1: "<=") → "<=" binary_operator(1: ">=") → ">=" binary_operator(1: "+") → "+" binary_operator(1: "-") → "-" binary_operator(1: "*") → "*" binary_operator(1: "DIV") → "DIV" binary_operator(1: "MOD") → "MOD"		binary_operator = tokenAND tokenOR tokenEQUAL tokenNOTEQUAL tokenLESSOREQUAL tokenGREATEROREQUAL tokenPLUS tokenMINUS tokenMUL tokenDIV tokenMOD;	{ LA_IS, {T_AND_0 }, {"binary_operator"}, {\
binary_operator = "AND" "OR" "==" "!=" "<=" ">=" "+" "-" "*" "DIV" "MOD";				binary_operator = tokenAND tokenOR tokenEQUAL tokenNOTEQUAL tokenLESSOREQUAL tokenGREATEROREQUAL tokenPLUS tokenMINUS tokenMUL tokenDIV tokenMOD;		{ LA_IS, {"["], {"index_action"}, {\

						{ LA_IS, { T_DIV_0 }, { "binary_operator", {\ { LA_IS, { "" }, 1, { T_DIV_0 } } }\ }};\ { LA_IS, { T_MOD_0 }, { "binary_operator", {\ { LA_IS, { "" }, 1, { T_MOD_0 } } }\ }};\ }
binary_action = binary_operator , expression;	binary_action = binary_operator , expression;	binary_action → binary_operator expression	binary_action(1: "AND", "OR", "=", "!=", "<=", ">=", "+", "-", "**", "DIV", "MOD") → binary_operator expression		binary_action = binary_operator >> expression;	{ LA_IS, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_OR_EQUAL_0, T_GREATER_OR_EQUAL_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, { "binary_action", {\ { LA_IS, { "" }, 2, { "binary_operator", "expression" } } }\ }};\ }
left_expression = group_expression unary_operation ident , [index_action] value cond_block;	left_expression = group_expression unary_operation ident , index_action__optional value cond_block;	left_expression → group_expression left_expression → unary_operation left_expression → ident , index_action__optional left_expression → value left_expression → cond_block	left_expression(1: "(") → group_expression left_expression(1: "NOT") → unary_operation left_expression(1: "-") → ident , index_action__optional left_expression(1: "0", "1", "2", "3", "4", "5", "6", "7", "8", "9") → value left_expression(1: "+", "-"; 2: "0", "1", "2", "3", "4", "5", "6", "7", "8", "9") → value left_expression(1: "IF") → cond_block	binary_action = binary_operator >> expression;	left_expression = group_expression unary_operation ident >> index_action__optional value cond_block;	{LA_IS, { "(" }, { "left_expression", {\ { LA_IS, { "" }, 1, { "group_expression" } } }\ }};\ {LA_IS, { T_NOT_0 }, { "left_expression", {\ { LA_IS, { "" }, 1, { "unary_operation" } } }\ }};\ {LA_IS, { "ident_terminal" }, { "left_expression", {\ { LA_IS, { "" }, 2, { "ident", "index_action__optional" } } }\ }};\ {LA_IS, { "unsigned_value_terminal" }, {\ left_expression", {\ { LA_IS, { "" }, 1, { "value" } } }\ }};\ {LA_IS, { T_ADD_0, T_SUB_0 }, { "left_expression", {\ { LA_IS, { "unsigned_value_terminal" }, 1, { "value" } } } }\ /*{LA_NOT, { "unsigned_value_terminal" }, 1, {\ "unary_operation" } } */\ }};\ {LA_IS, { T_IF_0 }, { "left_expression", {\ { LA_IS, { "" }, 1, { "cond_block" } } }\ }};\ }
	index_action__optional = index_action ε;	index_action__optional → index_action index_action__optional → ε	index_action__optional(1: "(") → index_action index_action__optional(1: "! "(") → ε		index_action__optional = index_action "";	{LA_IS, { "(" }, { "index_action__optional", {\ { LA_IS, { "" }, 1, { "index_action" } } }\ }};\ {LA_NOT, { "(" }, { "index_action__optional", {\ { LA_IS, { "" }, 0, { "" } } }\ }};\ }
expression = left_expression , binary_action__iteration;	expression = left_expression , binary_action__iteration;	expression → left_expression binary_action__iteration	expression(1: "(", "NOT", "+", "-", "_", "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "IF") → left_expression binary_action__iteration		expression = left_expression >> binary_action__iteration;	{LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", T_IF_0 }, {\ "expression", {\ { LA_IS, { "" }, 2, { "left_expression", "binary_action__iteration" } } }\ }};\ }
	binary_action__iteration = binary_action, binary_action__iteration ε;	binary_action__iteration → binary_action binary_action__iteration → ε	binary_action__iteration(1: "AND", "OR", "=", "!=", "<=", ">=", "+", "-", "**", "DIV", "MOD") → binary_action binary_action__iteration binary_action__iteration(1: "AND", "OR", "!=", "!=", "!="<=", "!=">=", "!="+", "!="-", "!="*", "!="DIV", "!="MOD") → ε		binary_action__iteration = binary_action >> binary_action__iteration "";	{LA_IS, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_OR_EQUAL_0, T_GREATER_OR_EQUAL_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, {\ "binary_action__iteration", {\ { LA_IS, { "" }, 2, { "binary_action", "binary_action__iteration" } } }\ }};\ {LA_NOT, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_OR_EQUAL_0, T_GREATER_OR_EQUAL_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, {\ "binary_action__iteration", {\ { LA_IS, { "" }, 0, { "" } } }\ }};\ }
group_expression = "(" , expression , ")";	group_expression = "(" , expression , ")";	group_expression → "(" expression ")"	group_expression(1: "(") → "(" expression ")"	group_expression = tokenGROUPEXPRESSIONBEGIN >> expression >> tokenGROUPEXPRESSIONEND;	group_expression = tokenGROUPEXPRESSIONBEGIN >> expression >> tokenGROUPEXPRESSIONEND;	{LA_IS, { "(" }, { "group_expression", {\ { LA_IS, { "" }, 3, { "(", "expression", ")" } } }\ }};\ }
expression_or_cond_block_with_optional_assign = expression , assign_to_right__optional;	expression_or_cond_block_with_optional_assign = expression , assign_to_right__optional;	expression_or_cond_block_with_optional_assign → expression assign_to_right__optional	expression_or_cond_block_with_optional_assign(1: "(", "NOT", "+", "-", "_", "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "IF") → expression assign_to_right__optional	expression_or_cond_block__with_optional_assign = expression >> -(tokenLRASSIGN >> ident >> -index_action);	expression_or_cond_block_with_optional_assign = expression >> assign_to_right__optional;	{LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", T_IF_0 }, {\ "expression_or_cond_block__with_optional_assign", {\ { LA_IS, { "" }, 2, { "expression", "assign_to_right__optional" } } }\ }};\ }
	assign_to_right = "=" , ident , index_action__optional;	assign_to_right → "=" ident index_action__optional	assign_to_right(1: "=") → "=" ident index_action__optional		assign_to_right = tokenLRASSIGN >> ident >> index_action__optional;	{LA_IS, { T_LRASSIGN_0 }, { "assign_to_right", {\ { LA_IS, { "" }, 3, { T_LRASSIGN_0, "ident", "index_action__optional" } } }\ }};\ }
	assign_to_right__optional = assign_to_right ε;	assign_to_right__optional → assign_to_right assign_to_right__optional → ε;	assign_to_right__optional(1: "=") → assign_to_right assign_to_right__optional(1: "! "=") → ε;		assign_to_right__optional = assign_to_right "";	{ LA_IS, { T_LRASSIGN_0 }, {\ "assign_to_right__optional", {\ { LA_IS, { "" }, 1, { "assign_to_right" } } }\ }};\ { LA_NOT, { T_LRASSIGN_0 }, {\ "assign_to_right__optional", {\ { LA_IS, { "" }, 0, { "" } } }\ }};\ }
if_expression = expression;	if_expression = expression;	if_expression → expression	if_expression(1: "(", "NOT", "+", "-", "_", "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "IF") → expression	if_expression = SAME_RULE(expression);	if_expression = SAME_RULE(expression);	{LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", T_IF_0 }, {\ "if_expression", {\ { LA_IS, { "" }, 1, { "expression" } } }\ }};\ }
body_for_true = block_statements;	body_for_true = block_statements;	body_for_true → block_statements	body_for_true(1: "(") → block_statements	body_for_true = SAME_RULE(block_statements);	body_for_true = SAME_RULE(block_statements);	{LA_IS, { T_BEGIN_BLOCK_0 }, {\ "body_for_true", {\ { LA_IS, { "" }, 1, { "block_statements" } } }\ }};\ }
false_cond_block_without_else = "ELSE" , "IF" , if_expression , body_for_true;	false_cond_block_without_else = "ELSE" , "IF" , if_expression , body_for_true;	false_cond_block_without_else → "ELSE" "IF" if_expression body_for_true	false_cond_block_without_else(1: "ELSE") → "ELSE" "IF" if_expression body_for_true	false_cond_block_without_else = tokenELSE >> tokenIF >> if_expression >> body_for_true;	false_cond_block_without_else = tokenELSE >> tokenIF >> if_expression >> body_for_true;	{LA_IS, { T_ELSE_IF_0 }, {\ "false_cond_block_without_else", {\ { LA_IS, { "" }, 4, { T_ELSE_IF_0, T_ELSE_IF_1, "if_expression", "body_for_true" } } }\ }};\ }
body_for_false = "ELSE" , block_statements;	body_for_false = "ELSE" , block_statements;	body_for_false → "ELSE" block_statements	body_for_false(1: "ELSE") → "ELSE" block_statements	body_for_false = tokenELSE >> block_statements;	body_for_false = tokenELSE >> block_statements;	{LA_IS, { T_ELSE_BLOCK_0 }, {\ "body_for_false", {\ { LA_IS, { "" }, 2, { T_ELSE_BLOCK_0, "block_statements" } } }\ }};\ }
cond_block = "IF" , if_expression , body_for_true, false_cond_block_without_else__iteration , body_for_false__optional;	cond_block = "IF" , if_expression , body_for_true, false_cond_block_without_else__iteration , body_for_false__optional;	cond_block → "IF" if_expression body_for_true body_for_true false_cond_block_without_else__iteration body_for_false__optional	cond_block(1: "IF") → "IF" if_expression body_for_true false_cond_block_without_else__iteration body_for_false__optional	cond_block = tokenIF >> if_expression >> body_for_true >> *false_cond_block_without_else >> -body_for_false;	cond_block = tokenIF >> if_expression >> body_for_true >> false_cond_block_without_else__iteration >> body_for_false__optional;	{LA_IS, { T_IF_0 }, {\ "cond_block", {\ { LA_IS, { "" }, 5, { T_IF_0, "if_expression", "body_for_true", "false_cond_block_without_else__iteration", "body_for_false__optional" } } }\ }};\ }
	false_cond_block_without_else__iteration = false_cond_block_without_else, false_cond_block_without_else__iteration ε;	false_cond_block_without_else__iteration → false_cond_block_without_else false_cond_block_without_else__iteration	false_cond_block_without_else__iteration(1: "ELSE"; 2: "IF") → false_cond_block_without_else false_cond_block_without_else__iteration		false_cond_block_without_else__iteration = false_cond_block_without_else >> false_cond_block_without_else__iteration "";	{LA_IS, { T_ELSE_IF_0 }, {\ "false_cond_block_without_else__iteration", {\ { LA_IS, { T_ELSE_IF_1 }, 2, {\ }

		false_cond_block_without_else__iteration → ε	false_cond_block_without_else__iteration(1: "ELSE"; 2: !"IF") → ε false_cond_block_without_else__iteration(1: !"ELSE") → ε			"false_cond_block_without_else", "false_cond_block_without_else__iteration" }}\
	body_for_false__optional = body_for_false ε;	body_for_false__optional → body_for_false body_for_false__optional → ε	body_for_false__optional(1: "FALSE") → body_for_false body_for_false__optional(1: !"FALSE") → ε		body_for_false__optional = body_for_false "";	{LA_IS, {T_ELSE_BLOCK_0}, { "body_for_false__optional",{\
cycle_begin_expression = expression;	cycle_begin_expression = expression;	cycle_begin_expression → expression	cycle_begin_expression(1: "(" , "NOT" , "+" , "-" , " " , "0" , "1" , "2" , "3" , "4" , "5" , "6" , "7" , "8" , "9" , "IF") → expression	cycle_begin_expression = SAME_RULE(expression);	cycle_begin_expression = SAME_RULE(expression);	{LA_IS, { "(" , T_NOT_0 , T_ADD_0 , T_SUB_0 , "ident_terminal" , "unsigned_value_terminal" , T_IF_0 } , { "cycle_begin_expression" ,{\
cycle_end_expression = expression;	cycle_end_expression = expression;	cycle_end_expression → expression	cycle_end_expression(1: "(" , "NOT" , "+" , "-" , " " , "0" , "1" , "2" , "3" , "4" , "5" , "6" , "7" , "8" , "9" , "IF") → expression	cycle_end_expression = SAME_RULE(expression);	cycle_end_expression = SAME_RULE(expression);	{LA_IS, { "(" , T_NOT_0 , T_ADD_0 , T_SUB_0 , "ident_terminal" , "unsigned_value_terminal" , T_IF_0 } , { "cycle_end_expression" ,{\
cycle_counter = ident;	cycle_counter = ident;	cycle_counter → ident	cycle_counter(1: " ") → ident	cycle_counter = SAME_RULE(ident);	cycle_counter = SAME_RULE(ident);	{LA_IS, { "ident_terminal" } , { "cycle_counter" ,{\
cycle_counter_lr_init = cycle_begin_expression , "=" , cycle_counter;	cycle_counter_lr_init = cycle_begin_expression , "=" cycle_counter	cycle_counter_lr_init → cycle_begin_expression "=" cycle_counter	cycle_counter_lr_init(1: "(" , "NOT" , "+" , "-" , " " , "0" , "1" , "2" , "3" , "4" , "5" , "6" , "7" , "8" , "9" , "IF") → cycle_begin_expression " =" cycle_counter	cycle_counter_lr_init = cycle_begin_expression >> tokenLRASSIGN >> cycle_counter;	cycle_counter_lr_init = cycle_begin_expression >> tokenLRASSIGN >> cycle_counter;	{LA_IS, { "(" , T_NOT_0 , T_ADD_0 , T_SUB_0 , "ident_terminal" , "unsigned_value_terminal" , T_IF_0 } , { "cycle_counter_lr_init" ,{\
cycle_counter_init = cycle_counter_lr_init;	cycle_counter_init = cycle_counter_lr_init;	cycle_counter_init → cycle_counter_lr_init	cycle_counter_init(1: "(" , "NOT" , "+" , "-" , " " , "0" , "1" , "2" , "3" , "4" , "5" , "6" , "7" , "8" , "9" , "IF") → cycle_counter_lr_init	cycle_counter_init = SAME_RULE(cycle_counter_lr_init);	cycle_counter_init = SAME_RULE(cycle_counter_lr_init);	{LA_IS, { "(" , T_NOT_0 , T_ADD_0 , T_SUB_0 , "ident_terminal" , "unsigned_value_terminal" , T_IF_0 } , { "cycle_counter_init" ,{\
cycle_counter_last_value = cycle_end_expression;	cycle_counter_last_value = cycle_end_expression;	cycle_counter_last_value → cycle_end_expression	cycle_counter_last_value(1: "(" , "NOT" , "+" , "-" , " " , "0" , "1" , "2" , "3" , "4" , "5" , "6" , "7" , "8" , "9" , "IF") → cycle_end_expression	cycle_counter_last_value = SAME_RULE(cycle_end_expression);	cycle_counter_last_value = SAME_RULE(cycle_end_expression);	{LA_IS, { "(" , T_NOT_0 , T_ADD_0 , T_SUB_0 , "ident_terminal" , "unsigned_value_terminal" , T_IF_0 } , { "cycle_counter_last_value" ,{\
cycle_body = "DO" , {statement} block_statements);	cycle_body = "DO" , statement__or__block_statements;	cycle_body → "DO" statement__or__block_statements	cycle_body(1: "DO") → "DO" statement__or__block_statements	cycle_body = tokenDO >> (statement block_statements);	cycle_body = tokenDO >> statement__or__block_statements;	{LA_IS, { T_DO_0 } , { "cycle_body" ,{\
forto_cycle = "FOR" , cycle_counter_init , "TO" , cycle_counter_last_value , cycle_body;	forto_cycle = "FOR" , cycle_counter_init , "TO" , cycle_counter_last_value , cycle_body;	forto_cycle → "FOR" cycle_counter_init "TO" , cycle_counter_last_value cycle_body	forto_cycle(1: "FOR") → "FOR" cycle_counter_init "TO" , cycle_counter_last_value cycle_body	forto_cycle = tokenFOR >> cycle_counter_init >> tokenTO >> cycle_counter_last_value >> cycle_body;	forto_cycle = tokenFOR >> cycle_counter_init >> tokenTO >> cycle_counter_last_value >> cycle_body;	{LA_IS, { T_FOR_0 } , { "forto_cycle" ,{\
	statement__or__block_statements = statement block_statements;	statement__or__block_statements → statement block_statements	statement__or__block_statements(1: !"{") → statement statement__or__block_statements(1: "(") → block_statements		statement__or__block_statements = statement block_statements;	{LA_IS, { "ident_terminal" , "(" , T_NOT_0 , "unsigned_value_terminal" , T_ADD_0 , T_SUB_0 , T_IF_0 , T_FOR_0 , T_INPUT_0 , T_OUTPUT_0 , T_SEMICOLON_0 } , { "statement__or__block_statements" ,{\
input_rule = "GET" , (ident , [index_action] "(" , ident , [index_action] , ")");	input_rule = "GET" , argument_for_input;	input_rule → "GET" argument_for_input	input_rule(1: "GET") → "GET" argument_for_input	input_rule = tokenGET >> {ident >> -index_action tokenGROUPEXPRESSIONBEGIN >> ident >> -index_action >> tokenGROUPEXPRESSIONEND};	input_rule = tokenGET >> argument_for_input;	{LA_IS, { T_INPUT_0 } , { "input_rule" ,{\
	argument_for_input = ident , index_action__optional; argument_for_input = "(" , "ident" , "index_action__optional" , ")" ;	argument_for_input → ident index_action__optional argument_for_input → "(" " ident" "index_action__optional" ")"	argument_for_input(1: " ") → ident index_action__optional argument_for_input(1: "(") → "(" " ident" "index_action__optional" ")"		argument_for_input = ident >> index_action__optional tokenGROUPEXPRESSIONBEGIN >> ident >> index_action__optional >> tokenGROUPEXPRESSIONEND;	{LA_IS, { "ident_terminal" } , { "argument_for_input" ,{\
output_rule = "PUT" , expression;	output_rule = "PUT" , expression;	output_rule → "PUT" expression	output(1: "PUT") → "PUT" expression	output_rule = tokenPUT >> expression;	output_rule = tokenPUT >> expression;	{LA_IS, { T_OUTPUT_0 } , { "output_rule" , {\
statement = expression_or_cond_block__with_optional_assign forto_cycle input_rule output_rule ";" ;	statement = expression_or_cond_block__with_optional_assign forto_cycle input_rule output_rule ";" ;	statement → expression_or_cond_block__with_optional_assign statement → forto_cycle statement → input_rule statement → output_rule statement → ";"	statement(1: "(" , "NOT" , "+" , "-" , " " , "0" , "1" , "2" , "3" , "4" , "5" , "6" , "7" , "8" , "9" , "IF") → expression_or_cond_block__with_optional_assign statement(1: "FOR") → forto_cycle statement(1: "GET") → input_rule statement(1: "PUT") → output_rule statement(1: ";") → ";"	statement = expression_or_cond_block__with_optional_assign forto_cycle input_rule output_rule tokensEMICOLON;	statement = expression_or_cond_block__with_optional_assign forto_cycle input_rule output_rule tokensEMICOLON;	{LA_IS, { "(" , T_NOT_0 , "ident_terminal" , "unsigned_value_terminal" , T_ADD_0 , T_SUB_0 , T_IF_0 } , { "statement" , {\
statement__iteration = statement, statement__iteration ε;	statement__iteration = statement, statement__iteration ε;	statement__iteration → statement statement__iteration → ε	statement__iteration(1: " " , "(" , "NOT" , "0" , "1" , "2" , "3" , "4" , "5" , "6" , "7" , "8" , "9" , "+" , "-" , "IF" , "FOR" , "GET" , "PUT" , ";") → statement statement__iteration statement__iteration(1: !" " , !" "(" , !" NOT" , !" 0" , !" 1" , !" 2" , !" 3" , !" 4" , !" 5" , !" 6" , !" 7" , !" 8" , !" 9" , !" + " , !" - " , !" IF" , !" FOR" , !" GET" , !" PUT" , !" ;") → ε		statement__iteration = statement >> statement__iteration "";	{ LA_IS, { "ident_terminal" , "(" , T_NOT_0 , "unsigned_value_terminal" , T_ADD_0 , T_SUB_0 , T_IF_0 , T_FOR_0 , T_INPUT_0 , T_OUTPUT_0 , T_SEMICOLON_0 } , { "statement__iteration" ,{\

	letter_in_upper_case = "A" "B" "C" "D" "E" "F" "G" "H" "I" "J" "K" "L" "M" "N" "O" "P" "Q" "R" "S" "T" "U" "V" "W" "X" "Y" "Z";	letter_in_upper_case → "A" letter_in_upper_case → "B" letter_in_upper_case → "C" letter_in_upper_case → "D" letter_in_upper_case → "E" letter_in_upper_case → "F" letter_in_upper_case → "G" letter_in_upper_case → "H" letter_in_upper_case → "I" letter_in_upper_case → "J" letter_in_upper_case → "K" letter_in_upper_case → "L" letter_in_upper_case → "M" letter_in_upper_case → "N" letter_in_upper_case → "O" letter_in_upper_case → "P" letter_in_upper_case → "Q" letter_in_upper_case → "R" letter_in_upper_case → "S" letter_in_upper_case → "T" letter_in_upper_case → "U" letter_in_upper_case → "V" letter_in_upper_case → "W" letter_in_upper_case → "X" letter_in_upper_case → "Y" letter_in_upper_case → "Z"	letter_in_upper_case(1: "A") → "A" letter_in_upper_case(1: "B") → "B" letter_in_upper_case(1: "C") → "C" letter_in_upper_case(1: "D") → "D" letter_in_upper_case(1: "E") → "E" letter_in_upper_case(1: "F") → "F" letter_in_upper_case(1: "G") → "G" letter_in_upper_case(1: "H") → "H" letter_in_upper_case(1: "I") → "I" letter_in_upper_case(1: "J") → "J" letter_in_upper_case(1: "K") → "K" letter_in_upper_case(1: "L") → "L" letter_in_upper_case(1: "M") → "M" letter_in_upper_case(1: "N") → "N" letter_in_upper_case(1: "O") → "O" letter_in_upper_case(1: "P") → "P" letter_in_upper_case(1: "Q") → "Q" letter_in_upper_case(1: "R") → "R" letter_in_upper_case(1: "S") → "S" letter_in_upper_case(1: "T") → "T" letter_in_upper_case(1: "U") → "U" letter_in_upper_case(1: "V") → "V" letter_in_upper_case(1: "W") → "W" letter_in_upper_case(1: "X") → "X" letter_in_upper_case(1: "Y") → "Y" letter_in_upper_case(1: "Z") → "Z"	a = "a"; b = "b"; c = "c"; d = "d"; e = "e"; f = "f"; g = "g"; h = "h"; i = "i"; j = "j"; k = "k"; l = "l"; m = "m"; n = "n"; o = "o"; p = "p"; q = "q"; r = "r"; s = "s"; t = "t"; u = "u"; v = "v"; w = "w"; x = "x"; y = "y"; z = "z"; letter_in_upper_case = A B C D E F G H I J K L M N O P Q R S T U V W X Y Z;	a = "a"; b = "b"; c = "c"; d = "d"; e = "e"; f = "f"; g = "g"; h = "h"; i = "i"; j = "j"; k = "k"; l = "l"; m = "m"; n = "n"; o = "o"; p = "p"; q = "q"; r = "r"; s = "s"; t = "t"; u = "u"; v = "v"; w = "w"; x = "x"; y = "y"; z = "z"; letter_in_upper_case = A B C D E F G H I J K L M N O P Q R S T U V W X Y Z;	\\
				STRICT_BOUNDARIES = (BOUNDARY >> *(BOUNDARY)) (!(qi::alpha qi::char_("_."))); BOUNDARIES = (BOUNDARY >> *(BOUNDARY) NO_BOUNDARY); BOUNDARY = BOUNDARY_SPACE BOUNDARY_TAB BOUNDARY_VERTICAL_TAB BOUNDARY_FORM_FEED BOUNDARY_CARRIAGE_RETURN BOUNDARY_LINE_FEED BOUNDARY_NULL; BOUNDARY_SPACE = " "; BOUNDARY_TAB = "\t"; BOUNDARY_VERTICAL_TAB = "\v"; BOUNDARY_FORM_FEED = "\f"; BOUNDARY_CARRIAGE_RETURN = "\r"; BOUNDARY_LINE_FEED = "\n"; BOUNDARY_NULL = "\0"; NO_BOUNDARY = ""; #define WHITESPACES \ STRICT_BOUNDARIES, \ BOUNDARIES, \ BOUNDARY, \ BOUNDARY_SPACE, \ BOUNDARY_TAB, \ BOUNDARY_VERTICAL_TAB, \ BOUNDARY_FORM_FEED, \ BOUNDARY_CARRIAGE_RETURN, \ BOUNDARY_LINE_FEED, \ BOUNDARY_NULL, \ NO_BOUNDARY	STRICT_BOUNDARIES = (BOUNDARY >> *(BOUNDARY)) (!(qi::alpha qi::char_("_."))); BOUNDARIES = (BOUNDARY >> *(BOUNDARY) NO_BOUNDARY); BOUNDARY = BOUNDARY_SPACE BOUNDARY_TAB BOUNDARY_VERTICAL_TAB BOUNDARY_FORM_FEED BOUNDARY_CARRIAGE_RETURN BOUNDARY_LINE_FEED BOUNDARY_NULL; BOUNDARY_SPACE = " "; BOUNDARY_TAB = "\t"; BOUNDARY_VERTICAL_TAB = "\v"; BOUNDARY_FORM_FEED = "\f"; BOUNDARY_CARRIAGE_RETURN = "\r"; BOUNDARY_LINE_FEED = "\n"; BOUNDARY_NULL = "\0"; NO_BOUNDARY = ""; #define WHITESPACES \ STRICT_BOUNDARIES, \ BOUNDARIES, \ BOUNDARY, \ BOUNDARY_SPACE, \ BOUNDARY_TAB, \ BOUNDARY_VERTICAL_TAB, \ BOUNDARY_FORM_FEED, \ BOUNDARY_CARRIAGE_RETURN, \ BOUNDARY_LINE_FEED, \ BOUNDARY_NULL, \ NO_BOUNDARY	\\
						\\ \\ \\ { LA_IS, { T_NAME_0 }, { "program____part1", {\n { LA_IS, { "" }, 7, { T_NAME_0, "program_name", T_SEMICOLON_0, T_BODY_0, T_DATA_0, "declaration_optional", T_SEMICOLON_0 }}\n }}}\n \\ },\n "program_rule"